

CampusFlow: arquitectura SaaS

Clase 13 | Resumen para estudiantes

CampusFlow inicia un backend para gestionar academias, usuarios y cursos. La clase se centra en dos decisiones: comprender el modelo SaaS y organizar el código por dominios de negocio.

Qué significa SaaS

Software as a Service es un modelo de entrega en el que el proveedor opera el software y lo ofrece como servicio por internet, habitualmente mediante suscripción. El cliente utiliza el servicio sin encargarse de instalar y mantener su infraestructura.

En CampusFlow, una academia es un cliente del producto. La organización del código ayuda a desarrollar sus capacidades, pero no implementa por sí sola el aislamiento de los datos de cada academia.

Monolito modular por dominio

usuarios y cursos agrupan capacidades del negocio. Cada dominio contiene sus propias capas. El sistema sigue siendo una sola aplicación Spring Boot y un único despliegue; separar paquetes no lo convierte en microservicios.

Organización conceptual del proyecto

```
com.devsenior.campusflow
  usuarios/
    controller/  service/  repository/  model/
  cursos/
    controller/  service/  repository/  model/
  common/
  config/
```

Capa	Responsabilidad
controller	Recibir HTTP y delegar al servicio.
service	Aplicar reglas y coordinar operaciones.
repository	Consultar y guardar datos.
model	Representar las entidades del dominio.
common / config	Piezas compartidas y configuración.

El dominio usuarios se construye durante el recorrido principal; cursos aplica el mismo patrón como extensión. Esta organización es una decisión de diseño contextual, no una convención única obligatoria.

Una funcionalidad vertical y sus entornos

Del endpoint a la base de datos

Recorrido de la consulta

```
GET /api/usuarios
-> UsuarioController
-> UsuarioService.listarUsuarios()
-> UsuarioRepository.findAll()
-> MySQL
```

El controlador recibe UsuarioService por constructor. El servicio recibe UsuarioRepository y delega la consulta. La entidad Usuario representa los datos iniciales: id, nombre y email. Colocar el servicio entre HTTP y persistencia permite incorporar reglas sin mezclarlas con las rutas.

Perfiles de desarrollo y producción

Archivo / ajuste	Propósito
application.properties	Configuración común del proyecto.
application-dev.properties	Configuración del entorno de desarrollo.
application-prod.properties	Variables externas y validación del esquema.
SPRING_PROFILES_ACTIVE	Selecciona el perfil desde el entorno.

Ejemplo de configuración del perfil de producción

```
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=false
```

validate comprueba el esquema; no crea las tablas. Los cambios del esquema requieren un proceso de migración. En desarrollo, update facilita el ejercicio, pero no debe confundirse con una estrategia completa de mantenimiento de la base.

Errores y límites del resultado

El manejo común de errores incluye ResourceNotFoundException y un handler global. Ante un bean no encontrado, revisa paquetes y anotaciones; ante una tabla inexistente, revisa esquema y perfil; ante un placeholder sin resolver, revisa las variables del entorno.

Pendiente del producto: un tenant representa una academia cuyos datos deben aislarse de otras. Esta clase no implementa multitenancy, pagos ni despliegue. Sus entregables son la estructura modular y el dominio inicial; no un SaaS terminado.

Material base: guía de clase 13, módulo 4. Ana Alvarado, Dev Senior.